

DevOps GPT - For deployment and Failure Detection

Anuj Pradip Deshmukh, Vedant Ritesh Agrawal, Vighnesh Vivek Ise
and Prof. Deepali V Shinkar

Department of Artificial Intelligence and Data Science
K. K. Wagh Institution of Engineering Education and Research
Nashik, India

{apdeshmukh371122, vragrawal371122, vvise371123, dvshinkar}@kkwagh.edu.in

Abstract—These days, we live in a superfast technological era where we are always being pressured to deliver software that can be made faster and at the same time ensure that it is rock solid, it can scale properly and we can watch what is going on in real time. Classic DevOps was wonderful, and it is frequently unable to cope with the madness of current cloud-native stacks. The project is being implemented due to a straightforward yet rugged question why could not DevOps pipelines be more intelligent, autonomous, and durable? The answer? New AI-based automation architecture based on the Model Context Protocol (MCP). MCP collects and digests distributed cloud infra operational data with smart agents and big language models allowing the system to identify patterns, interpret logs in context, and even make decisions on the fly and not require constant human supervision. The large-scale benefit of this framework is its ability to uplift DevOps teams, in terms of business. It provides proactive error detection, automatic rollback and intelligent remediation that reduce downtimes, and enhance the speed of deployment. Team members now have the ability to predict and correct issues early reducing the mean time to recovery and enhancing system resilience.

Index Terms—DevOps, Artificial Intelligence (AI), Model Context Protocol (MCP), CI/CD, Anomaly Detection, Automation, Real-Time Monitoring, Machine Learning, Log Analysis, Observability, Reinforcement Learning, Infrastructure as Code (IaC).

I. INTRODUCTION

We are living in a world of rapid release patterns in the current software development environment competitive markets and evolving user demands keep us on our toes. That is why Devops and CI/CD pipelines have become a common practice by companies in an attempt to automate the commit-to-production path. These approaches have visibly increased frequency of deployment and operational responsiveness, yet they have complicated, in particular, microservices-based, cloud-native designs [1]. One of the huge burdens in contemporary settings is overwhelming data volume and pace of operations. Distributed system logs, metrics and traces are accumulated, and real-time analysis and proactive failure detection becomes a massive human headache. This causes the majority of the DevOps pipelines, despite automation, to remain reactive. In case of a wrong deployment, configuration drift or performance degradation, engineers need to manually find the root causes of the issue, leading to lengthy downtimes, increased MTTR, and wasting engineering resources [2]. This chokepoint completely disarm the DevOps: reliability and

agility. Pain points are what the emerging branch of AI in IT Operation (AIOps) desires to address by smacking AI in operations processes. The latest advances in LLM, in particular, GPT series, are providing us with the capabilities of NLP and pattern-recognition like never before [3]. These models have the ability to make sense of unstructured logs, join events across sources and emit readable information. Auto workflow generation and analysis of logs are some of the early uses of LLMs [4]. The question of the reliability of AI-generated outputs is also being addressed by others who suggest risk-assessment models of LLM code [5]. Yet these standalone AI systems have a long way to go before they are deployed in a more systematic and integrated use in live CI/CD loops. The existing solutions will not provide a single data orchestration layer to put real-time data into context and a concrete framework to transform AI insights into safe, automated actions in the DevOps toolchain- that is the research gap that we are hoping to cover.

II. RELATED WORK

DevOps-GPT framework is based on and overlaps with a number of research fields: DevOps evolution, AIOps emergence, and LLMs in software engineering. This part of the review of the literature identifies the particular gap that we are filling. The fundamental principles of Devops have transformed the way software is delivered with a shift to infrastructure in the integration of development and operations with an emphasis on automation, continuous integration and continuous delivery. The toolbox is now mature and is the reason why Ci/CD automation is becoming a mainstream practice. Jenkins has served as the choice of automation of building, testing and deploying phases. Tools such as Terraform are also used as IaC tools to provide environments in an automated manner. The speed and reliability of structured automation were demonstrated by Saxena et al. [1], who described a powerful DevOps pipeline, including IaC and pipeline-as-code. The de facto standard on controlling cloud-native apps is containerization and orchestration (Docker, Kubernetes) which provides us with scalable environments that are isolated. These tools, though, operate by fixed rules, without cognitive ability to react to unforeseen failures or and make processes more efficient depending on context. The emergence of AIOps

overcame the constraints of traditional tooling with regard to scale and complexity. Machine learning and data science are applied to IT operations issues with AIOps. Studies have examined the anomaly detection, predictive monitoring, and the analysis of logs. Numerous studies applied traditional ML models to identify the existence of outliers in time-series measurements provided by tools such as Prometheus to identify the presence of performance degradation early.

DevOps-GPT will be aimed at filling that gap. It builds upon the automation base at Jenkins and Terraform, takes on the proactive monitoring culture of AIOps and uses LLMs to get contextual insight, all overlaid with a safety-first culture that is inspired by risk-assessment models. New reinforcement-learning projects in resource management have demonstrated a potential to scale autonomously, but they have primarily been independent systems, and not components of a larger deployment automation stack. Unifying these AI capabilities into a system that executes throughout the deployment lifecycle is the most important contribution of DevOps-GPT over the previous work.

Log analysis usually consists of matching patterns and a number of pre-existing rules, and previous anomaly detection studies have mostly focused on metric-based algorithms based on the application of traditional machine learning models. The interesting thing about DevOps-GPT is that it integrates the two ideas with a large language model that can understand natural language, enabling more contextual and adaptable failure detection [2], [3]. Furthermore, even though most AIOps solutions excel at monitoring and notifications, they typically don't integrate into the deployment line itself, so they can't truly carry out fully autonomous fixes [4].

III. SYSTEM ARCHITECTURE AND DESIGN

The DevOps-GPT structure is set up to produce an intelligent, closed-loop system that seamlessly integrates with the current DevOps toolchains and offers a certain amount of cognitive capacity to function completely on its own. Real-time data context, analysis written by AI software, and automated fixes are the main sensory impressions, and they all work within a scalable feedback-based architecture.

A. High- Level Architecture

Built on a multi-layered architecture, the DevOps-GPT framework combines traditional DevOps tools with AI-based analytics to create a self-learning system. To have a complete understanding, the entire set of data is collected from different sources. CI/CD software such as GitHub actions and GitLab CI continue providing us with build and live deploy status. Meanwhile, infra platforms such as Kubernetes, Docker, and Terraform emit in orchestration events and provisioning logs with the information about what is going on in the environment. Performance metrics and time-series information of monitoring tools such as Prometheus and Datadog are augmented with this ops data and monitor health and resources. Version control also provides us with lots of developmental

background- mainly Git repos that provide us with complete commit histories and code modifications.

The main component of the design is the Model Context Protocol (MCP), which is a single data bus. It captures all the various streams of data and standardizes them into a single clean structure schema. One important component is the context enrichment where the raw data is supplemented with deployment IDs, service tags and other data about the environment. In absence of such enrichment, we cannot obtain a unified, context rich picture of the health and behaviour of systems, which is necessary in order to have the AI engines engage in accurate and correlated analysis of all.

B. Technical Stack

- i. Core Protocol: Model Context Protocol (MCP) of real-time aggregation of data and context of distributed cloud systems, the primary orchestration layer.
- ii. AI/ML Components: GPT-3.5/4 (OpenAI) to analyse logs, identify anomalies, and find root causes using natural language; TensorFlow/PyTorch to identify statistical outliers in performance data using models.
- iii. DevOps Automation: Jenkins to coordinate CI/CD pipelines and deployment automation; Terraform to provision IaC; Kubernetes and Docker to coordinate and manage containers and environments.
- iv. Data Processing: Log parsing/metric extraction custom units; real-time ingestion modules of CI/CD events; feedback to keep the model continuously improving.

C. System Data Flow

- i. Data Processing and Transformation: MCP ingests and contextualizes real-time data in Jenkins, Kubernetes, Terraform, and Prometheus and converts logs and metrics to the common schema.
- ii. AI Analysis and Detection: The AI (LLM) (GPT 3.5/4) will search through logs to identify patterns of failures, and ML models will identify anomalies in performance statistics and metrics.
- iii. Decision Generation and Action Planning: The system automatically generates responses such as rollback triggers, scaling moves and severity-ranked alerts (P0-3) to ops teams.
- iv. Implementation and Adoption: Approved actions are executed using Jenkins APIs, Kubernetes orchestration, and Terraform workflows, but they do not disrupt the existing pipelines.
- v. Feedback and Continuous Learning: The results of resolutions are reflected in the body of knowledge, which has the effect of refining the AI models and refining detection thresholds in a continuous learning cycle.

D. Innovation and Uniqueness

- i. Unifies Jenkins, Kubernetes, Terraform, and LLMs into a workflow.
- ii. Makes forecasts of deployment hiccups based on historical trends prior to their occurrence.
- iii. Continues to improve on detection through feedback and closed cases.

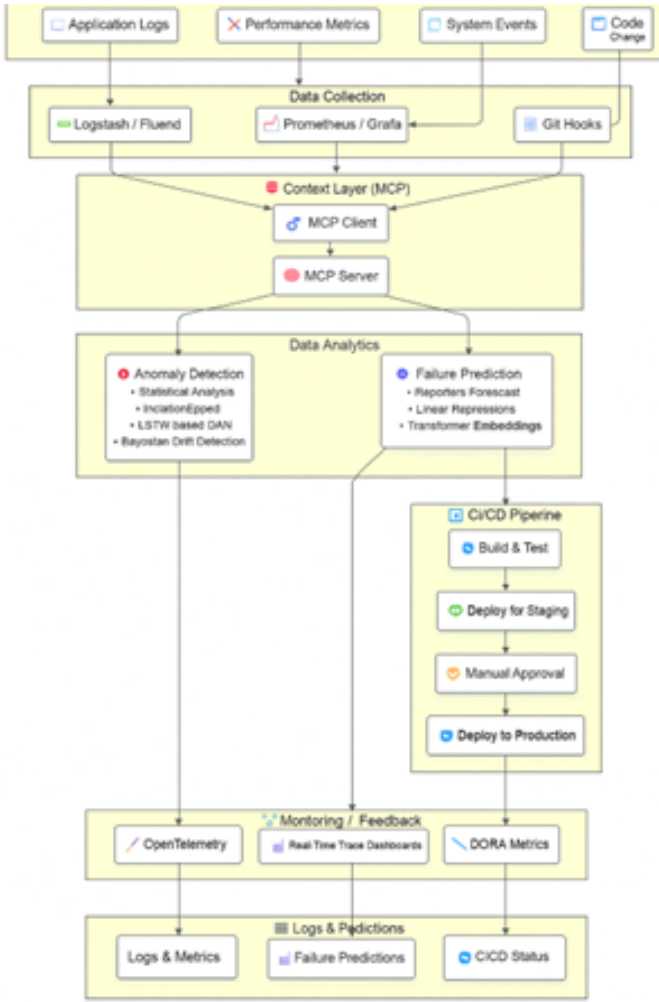


Fig. 1. System Architecture/Flow of DevOps GPT - for Deployment and failure detection

- iv. Uses NLP to decode stack traces and complicated error messages.
- v. Intelligent, automated decisions on safe, accurate roll-backs on deployment.
- vi. Automatically adjusts thresholds of alerts according to the system activity and false-positives.

IV. IMPLEMENTATION DETAILS

It is implemented with the Model Context Protocol as the central point of data exchange, which is connected in real-time to Jenkins, Kubernetes, and Terraform APIs. A Python service scrubs received streams of data, and sends it to the LLM analysis module, where GPT-4 searching through logs with tightly controlled prompts generates failure analysis. Automated actions can be run in the system with special API clients that directly communicate with DevOps tools so that you can roll back Jenkins jobs, scale Kubernetes, and modify Terraform configs. The outcome of interventions is captured in a feedback loop and the resolution patterns are stored in a

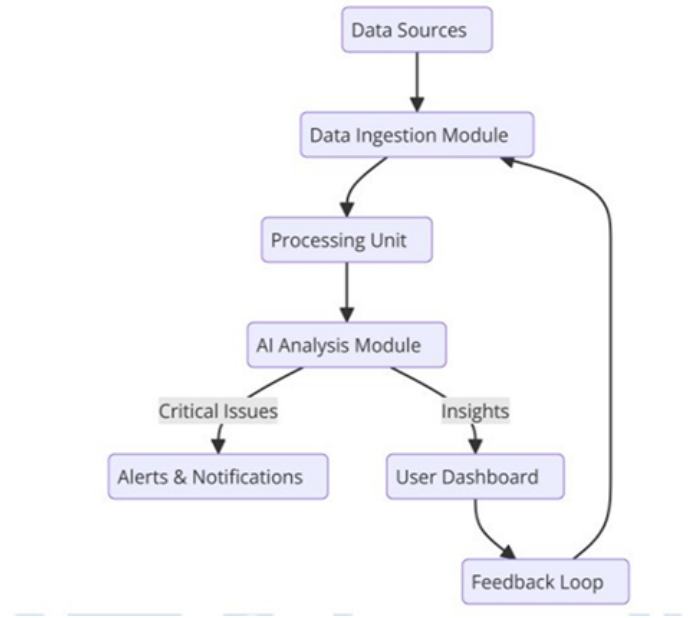


Fig. 2. Fig 2: Backend Components DevOps - GPT: For Deployment and Failure Detection

vector database, and continually refines the knowledge of the LLM within a Docker-based system run by Kubernetes.

A. Backend Components

DevOps-GPT code is structured around a Model Context Protocol Server, which is written in Python in the `asyncio` programming language. It is the real-time data aggregator which communicates directly with Jenkins, Kubernetes and Terraform APIs. This MCP server continues to stream running operational data to a specific Data Processing Service, logs and metrics are normalized with bespoke parsers and schema checks to ensure that all is clean and constant. The core analysis engine is the central component of the system, which receives GPT-4 through the OpenAI API and interprets DevOps logs with highly adjusted prompts. It will also incorporate a two-step pre-training system to perform a deep risk assessment of all AI-generated replies to ensure that everything remains dependable and safe.

The architecture has execution modules that are implemented with REST API clients to manage Jenkins jobs and automatically roll back and stop pipelines when the AI identifies failures. Meanwhile, Kubernetes operators manage container recovery and automatic scaling and Terraform controllers repair infrastructure through GitOps processes. The subsystem of feedback contains the results of resolution and the metrics of MTTR, which are continuously updated with a vector embedding in the knowledge base. This will continue to educate the system on the occurrences which will further our desired objective of reducing manual work and enhancing system resilience.

DevOps-GPT frontend is designed using simple web technology- HTML5 and CSS3 and Tailwind CSS to ensure a uniform appearance and feel and accessibility of the interface

to all user interactions. This clean installation produces a user friendly and integrated user experience without extraneous junk.

B. Frontend Framework

- i. DevOps-GPT frontend has been developed using simple web technologies: HTML 5 and CSS3.
- ii. Tailwind CSS also maintains everything to look good, themed and availed to all users.

V. EVALUATION AND RESULTS

This section presents a comprehensive evaluation of DevOps-GPT, including the test environment, performance assessment, software testing outcomes, accuracy analysis, and scalability observations. All content originally separated into subsections has been consolidated here for clarity and conciseness.

Test Environment: DevOps-GPT was evaluated using a controlled DevOps/SRE-oriented environment consisting of an Intel Core i5-10400 CPU, 16GB RAM, a 256GB SSD, and a 100 Mbps broadband connection. The system ran Ubuntu 22.04 LTS and used Python 3.10, FastAPI, Uvicorn, and the Mistral AI SDK. The dataset included 50+ real-world log files from Nginx, PostgreSQL, authentication systems, and application logs, ranging from 100 to 10,000 lines. Logs contained diverse issues such as HTTP failures, database errors, authentication problems, and performance bottlenecks.

Performance Parameters: Evaluation focused on five key performance indicators:

- Response time for pattern-based and AI-based log analysis
- Accuracy of severity classification and error detection
- System stability and fallback reliability during API failures
- Scalability under concurrent user load
- UI responsiveness and ease of use

Software Testing Overview: A total of 20 test cases were executed covering functionality, reliability, performance, and user experience. All 20 tests passed successfully, demonstrating system stability. Key findings include:

- Full 100% pass rate across functionality and API interactions
- Seamless fallback to pattern-matching during AI/API failures within 100ms
- No crashes, data corruption, or service interruptions
- UI response consistently under 200ms on desktop and mobile devices

Performance Metrics: Pattern-based analysis consistently completed within 1–2 seconds, offering rapid results. AI-powered analysis averaged 5 seconds due to external API latency while providing more detailed insights. Backend

processing times remained under 100ms, and the frontend maintained sub-150ms interactions even during load. Memory usage averaged 220MB, and CPU utilization generally remained below 40%, with occasional spikes during very large log processing.

Accuracy Evaluation: System accuracy was measured by comparing automated analysis with expert manual evaluation. AI mode achieved:

- 94.5% accuracy in severity classification
- 96.8% accuracy in error detection

Pattern-matching mode also performed reliably with 87.2% classification accuracy. Both modes demonstrated low false-positive and false-negative rates, ensuring dependable output for DevOps and SRE workflows.

Scalability Findings: DevOps-GPT demonstrated strong scalability. Up to 10 concurrent users showed no performance deterioration. Beyond this, the system maintained over 94% success rate for 20–50 users. The hybrid architecture—local processing plus optional AI inference—contributed to stable performance even when external APIs were under heavy load.

Discussion: Overall results show that DevOps-GPT significantly improves log analysis efficiency and operational readiness. Key strengths include:

- **Hybrid Intelligence:** AI provides contextual insights while pattern matching ensures offline dependability.
- **High Accuracy:** Automated severity detection and error recognition closely match expert analysis.
- **Fast Processing:** Analysis times reduced from minutes (manual) to seconds.
- **User-Friendly UI:** Clean interface, minimal latency, actionable recommendations.
- **Cross-Domain Capability:** Performs well on application, server, database, and security logs.

Compared to traditional methods, manual log inspection often requires 15–30 minutes, while grep-based searches require 2–3 minutes. DevOps-GPT consistently completes analysis within 2 seconds (pattern mode) or 5 seconds (AI mode), dramatically reducing Mean Time to Detect (MTTD) and Mean Time to Resolve (MTTR).

VI. DISCUSSION

DevOps-GPT presents a new twist to the intelligence of CI/CD pipeline by embedding large language models in the deployment process through the Model Context Protocol. In comparison with older DevOps tools, this configuration enables the system to discover the reason behind something going wrong in real time, determine the optimal solution and execute it automatically. It combines the information in logs, metrics, and infrastructure state and feeds that into an LLM and ultimately provides some degree of operational visibility that other automation tools have never been capable of matching.

VII. CONCLUSION AND FUTURE SCOPE

DevOps-GPT is, in fact, demonstrating how smooth it can be to intersperse big language models with DevOps-related stuff, in particular, with this Model Context Protocol thing. It addresses all those exasperating CI/CD hiccups, as it identifies the failures as they occur, calculates the reasons why they do, as well as corrects them automatically. In essence, it is a transition between old-school automation and AI smarts, thus, we witness fewer hiccups, less human hand-touching, and reduced get-back-up times. The evidence is in the fact that LLMs gnash their teeth on those ops data and actively make the system harder and at the same time, they do not conflict with typical DevOps tools. It provides companies with an actual, viable route to self-healing configuration and more intelligent, data-driven rollouts.

On the other hand, the following measures are rather exciting. It is planned to push the framework to multi-clouds and hybrid deployments, in order to be able to use the same code on other clouds. It may be possible to sharpen the spot-on detections further with a bit of domain specific fine-tuning of the various deployment patterns. Another terrific layer is to add predictive analytics of sizing and movement of resources in advance, which is in essence the prevention of problems before they blow up. Another study underway on how we talk to AI when the system is making its decisions would also increase trust and transparency, particularly in mission-critical prod setups. Federated learning exploration would assist in preserving data safety and training good models across companies. And lastly, making the system audit trail ready and compliance ready by beefing up the security side would prepare the system towards regulated industries.

REFERENCES

- [1] A. Saxena, S. Singh, S. Prakash, T. Yang, and R. S. Rathore, "DevOps Automation Pipeline Deployment with IaC (Infrastructure as Code)," arXiv preprint arXiv:2503.16038, Mar. 2025.
- [2] D. Mehta, "DevOps-GPT: Auto-Generating GitHub Actions Workflows using GPT-3.5/4," unpublished manuscript, [n.d.].
- [3] A. Goh, "Assessing LLM-Generated Code through Internal State Analysis: A Two-Stage Pretraining Framework," unpublished manuscript, [n.d.].
- [4] S. Qiu, Y. Zhang, T. Wang, and L. He, "A Review on the Advancements in Large Language Models: Implications for DevOps Automation," unpublished manuscript, [n.d.].
- [5] OpenAI, "Model Context Protocol (MCP) Documentation," [Online]. Available: <https://platform.openai.com/docs/mcp>
- [6] "Jenkins Automation Server Documentation," [Online]. Available: <https://www.jenkins.io/doc/>
- [7] "Kubernetes Production-Grade Container Orchestration," [Online]. Available: <https://kubernetes.io/docs/>
- [8] HashiCorp, "Terraform Infrastructure as Code," [Online]. Available: <https://www.terraform.io/docs>
- [9] "Prometheus Monitoring System," [Online]. Available: <https://prometheus.io/docs/>
- [10] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.
- [11] Gartner Research, "AIOps Platform Features and Capabilities," 2024.
- [12] Cloud Security Alliance, "Best Practices for CI/CD Pipeline Security," 2024.
- [13] "Large Language Models for Log Analysis: A Comprehensive Study," IEEE Transactions on Software Engineering, vol. 49, no. 3, pp. 1125–1140, Mar. 2024.